

INDIA.RUNS

Build what next India runs on

Team Name : Shortlist

Team Leader Name : Nandish S (solo)

Problem Statement : Intelligent Candidate Discovery & Ranking. From 100,000 candidates, surface the top 100 best-fit profiles for one Senior AI Engineer role, ranked by genuine fit rather than keyword overlap.

Solution Overview

- We rank candidates the way an experienced recruiter would: by what they have actually built, not by which keywords happen to appear on a profile.

Transparent core. Each final score is `relevance_core` x a behavioral-availability factor x a honeypot factor, so every rank breaks down into named numbers we can read and defend.

Semantic recall, capped. A TF-IDF JD-similarity term (weight 0.12) helps surface strong profiles written in plain language, but is weighted low on purpose so it can never override real career evidence.

Keyword-proof. Stuffing a profile with AI terms does nothing, because a skill earns credit only when the work history corroborates it.

Trap-aware and explainable. It separates genuine seniors from stuffers, look-alike 'behavioral twins' and planted impossible profiles, and no model runs at scoring time, so there is nothing to hallucinate. It finishes in about 55 seconds on a CPU.

JD Understanding & Candidate Evaluation

- **Requirements we extracted from the JD.** 5 to 9 years of experience (ideally 6 to 8); hands-on production work in ranking, search, recommendation or retrieval; strong Python and modern ML (PyTorch, embeddings, RAG, vector search); real offline evaluation (NDCG, A/B testing); a product-company rather than pure-services background; and being based in India or open to relocating.

Signals we weight most. Evidence of genuine information-retrieval work, skills we can actually trust, and whether the candidate sits inside the target experience band; availability signals then decide between otherwise-equal profiles.

How we go beyond keywords. A skill counts only when the career history backs it up; semantic similarity surfaces excellent candidates who never write buzzwords; and suspicious profiles are caught by internal contradictions, not by what they claim about themselves.

Ranking Methodology

- **Retrieve.** We stream and score all 100,000 candidates with no lossy pre-filter, so recall stays at 100% (the pool is small enough to score in full within the time budget).

Score. `relevance_core` is a weighted sum of named signals: career evidence 0.30, semantic JD-similarity 0.12, trusted skills 0.18, experience band 0.12, Python and evaluation experience 0.12, location 0.12, and education 0.04.

Adjust. We multiply by a behavioral-availability factor (0.85 to 1.05) from recency, recruiter response, notice period and open-to-work status, and by a honeypot factor that pushes internally impossible profiles down.

Rank. We sort by the final score and break ties deterministically by `candidate_id`, then take the top 100. Because every signal is combined arithmetically, we can explain exactly why any candidate sits where they do.

Explainability & Data Validation

- **Explanations that match the score.** Each candidate's reasoning is generated from the exact feature values that produced their rank, so the explanation can never disagree with it.

No hallucination, by construction. No language model runs at scoring time (the network is off), and every clause must cite a real field such as title, years, trusted skills, product industry or notice period.

Honest about weaknesses. We surface concerns, not just strengths: low recruiter response, long inactivity, onsite-only-abroad or a long notice period are stated plainly.

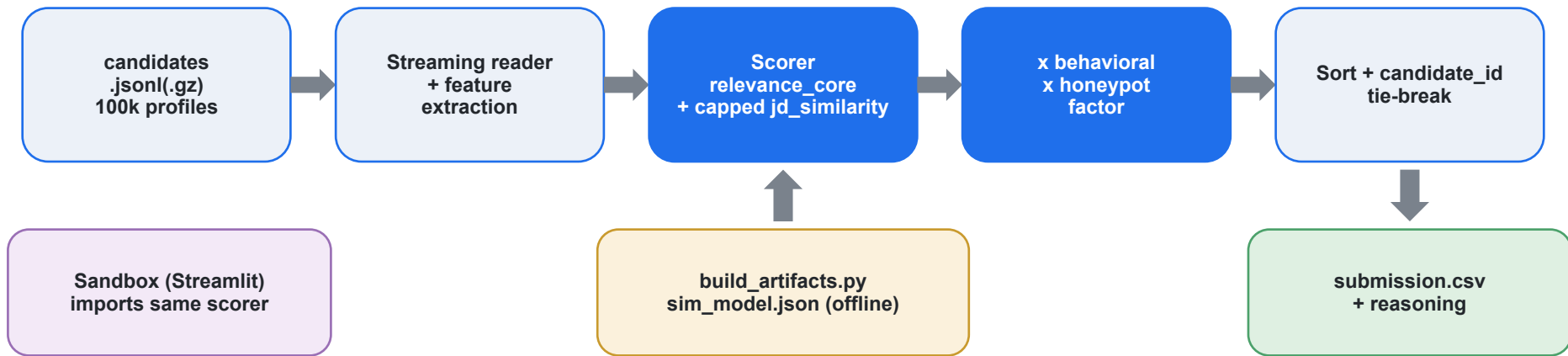
Incomplete data handled fairly. Sentinel values (a -1 GitHub or offer-history score) are treated as 'missing', never as the worst case, so a candidate is not punished for an unlinked account.

Suspicious profiles gated. A conjunctive honeypot gate needs at least two independent impossibility signals before it acts, so genuine seniors are never wrongly demoted; the result is 0% honeypots in the top 100 against a 10% disqualification line.

End-to-End Workflow

- **1. Offline precompute (untimed).** build_artifacts.py streams the pool and builds a TF-IDF table, four JD seed vectors and a data-derived calibration anchor, saved as sim_model.json (152 KB, committed).
- 2. Ranking (timed, under 5 minutes).** rank.py streams every candidate, scores each one (relevance_core + behavioral + honeypot), sorts with the candidate_id tie-break, and writes the top-100 CSV with a reasoning string per candidate.
- 3. Validation.** The official validator runs alongside our self-audit, which checks the honeypot rate, ID membership in the pool, output format and score distribution before any submission.
- 4. Live demo, same code.** The hosted sandbox imports the very same scorer module, so the interactive demo can never drift from what the grader reproduces.

System Architecture



The ranking step uses only the Python standard library: no GPU, no network, no model download. The committed 152 KB similarity model is the only precomputed artifact.

Results & Performance

- **Strong, in-band head.** The top 10 are all in-band IR/ML engineers at product companies (Search, Recommendation, NLP, ML and AI Research roles, 6 to 9 years); strong plain-language profiles surface even without buzzwords.

Stuffers do not survive. Of 5,588 keyword stuffers detected in the pool, none reach the top 100 (the best lands only at rank 628).

Traps handled. 0% honeypots in the top 100 (against a 10% disqualification line), and 100% recall on the hand-checked golden archetypes.

Comfortably within every constraint. About 55 seconds on 100k candidates (limit 5 minutes), well under 16 GB RAM, CPU-only, no network, and a ~150 KB model (limit 5 GB disk). It reproduces from a single command on a clean checkout.

Technologies Used

- **Python standard library only (ranking step).** json, gzip, csv, re and math. Zero ML dependencies means the timed step is trivially reproducible in the grader's Docker, fast, and free of version or download risk.

Custom stdlib TF-IDF (semantic layer). No scikit-learn and no model download. We chose this because the dataset's role descriptions are templated, so heavyweight embeddings add cost and reduce explainability without adding signal; a judge-panel comparison scored the transparent engine above an embeddings hybrid.

Streamlit. Powers the hosted sandbox demo only; it is never imported by the ranking step.

Git. Used for authentic, incremental version history that documents how the solution was built.

Submission Assets

GitHub repository. <https://github.com/Nandish3010/redrob-candidate-ranker>

Hosted sandbox. <https://redrob-candidate-ranker-ns.streamlit.app/>

Ranked output. outputs/submission.csv (top 100; columns candidate_id, rank, score, reasoning).

Single reproduce command. `python rank.py --candidates ./data/candidates.jsonl --out ./outputs/submission.csv`

 redrob | 

INDIA.RUNS

Build what next India runs on

THANK YOU